Architecture Logicielle - Atelier Final

Réponses aux demandes

Partie I - Mise en jambes

1. Architecture logicielle et rôle de l'architecte logiciel

L'architecture logicielle correspond à l'organisation structurante d'un système : composants, responsabilités, interactions, technologies, règles de sécurité, contraintes de performance, déploiement, maintenabilité et évolutivité.

Le rôle de l'architecte logiciel est de transformer les besoins métier et techniques en une solution cohérente. Il définit les choix structurants, découpe le système, fixe les règles de communication entre composants, anticipe les risques et garantit que l'application reste maintenable, sécurisée et scalable.

2. Design patterns : définition, utilité et exemples

Un design pattern est une solution de conception réutilisable pour un problème fréquent. Il ne s'agit pas d'un code prêt à copier, mais d'une manière éprouvée d'organiser les responsabilités entre objets, classes ou modules.

Utilité : améliorer la lisibilité, réduire le couplage, éviter la duplication, faciliter les tests, rendre le code plus maintenable et préparer les évolutions.

Pattern	Principe	Cas d'utilisation
Builder	Construire progressivement un objet ou une structure complexe.	Construction dynamique de requêtes SQL : select, from, where, orderBy, build.
Repository	Isoler l'accès aux données derrière une interface métier.	Un service de commande récupère une commande via OrderRepository sans connaître SQL.
Observer	Notifier automatiquement plusieurs composants lorsqu'un événement se produit.	Après création d'une commande : notification livreur, email client, journalisation.

3. Anti-pattern

Un anti-pattern est une mauvaise solution récurrente à un problème de conception. Il peut sembler pratique au départ, mais il dégrade généralement la maintenabilité, la testabilité et l'évolutivité.

Anti-pattern	Description	Conséquence
God Object	Une classe ou un fichier concentre trop de responsabilités.	Modifications risquées, tests difficiles, forte dette technique.
Spaghetti Code	Code désorganisé avec dépendances et conditions partout.	Compréhension difficile et évolution coûteuse.
Big Ball of Mud	Application sans structure claire, où tout dépend de tout.	Maintenance lente, bugs fréquents, refonte progressive nécessaire.

4. Caractéristiques, forces et faiblesses des architectures

Architecture	Caractéristiques	Forces	Faiblesses
Monolithique	Application regroupée dans un seul bloc déployable.	Simple au départ, déploiement direct, debug plus simple, adaptée à un MVP.	Couplage fort possible, scalabilité globale, déploiements risqués, maintenance difficile si le code est non structuré.
En couches	Séparation par responsabilités : présentation, application,	Lisibilité, maintenabilité, testabilité, séparation claire des responsabilités.	Peut devenir rigide ou verbeuse, dépendances mal gérées possibles.

	domaine, infrastructure, données.		
SOA / Microservices	Système découpé en services autonomes communiquant par API ou messages.	Scalabilité fine, déploiements indépendants, isolation des domaines métier.	Complexité opérationnelle, transactions distribuées, observabilité obligatoire, tests plus difficiles.
Event-driven	Les traitements sont déclenchés par des événements métier.	Découplage, temps réel, scalabilité, traitements asynchrones, bonne adaptation aux notifications.	Débogage plus complexe, cohérence éventuelle, besoin d'idempotence et de fiabilité des messages.

5. Couches dans une architecture en couches

Dans une architecture en couches, chaque couche représente un niveau de responsabilité. Une couche supérieure utilise les services de la couche inférieure sans mélanger les rôles.

Couche	Responsabilité
Présentation	Interfaces utilisateur, formulaires, pages web, affichage.
Application	Cas d'utilisation : créer une commande, accepter une livraison, ouvrir un litige.
Domaine / métier	Règles métier, entités, invariants, statuts, validations métier.
Infrastructure	Base de données, emails, notifications, GPS, fichiers, exports.
Données	Stockage relationnel, tables, index, contraintes, migrations.

Exemple : dans l'application de livraison, un contrôleur reçoit une demande de création de commande, appelle un cas d'utilisation applicatif, qui applique les règles métier via le domaine, puis persiste les données via un repository d'infrastructure.

Partie II - Modélisation d'application

1. Types d'utilisateurs

Utilisateur / acteur	Interactions avec l'application
Client	Consulte les catalogues, passe commande, suit la livraison en temps réel, ouvre un litige.
Fournisseur	Publie et met à jour ses catalogues, articles, prix et disponibilités.
Livreur	Reçoit les commandes, accepte ou refuse une livraison, met à jour les statuts, transmet sa position.
Administrateur	Supervise l'activité, gère les litiges, consulte les indicateurs, exporte les données.
Traqueur GPS	Envoie les positions associées à une livraison ou à un colis équipé.
Service de notification	Envoie les notifications email, SMS ou push aux clients, livreurs et administrateurs.

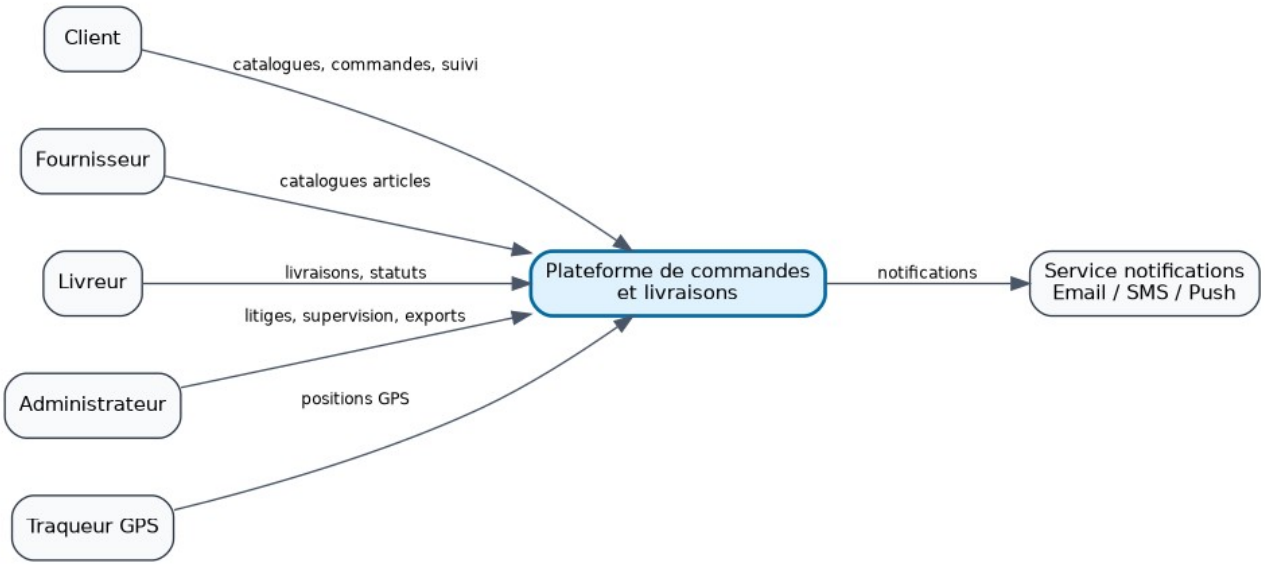
2. Architecture proposée et diagramme de contexte

Architecture proposée : monolithe modulaire sécurisé, basé sur une séparation en couches et une logique proche de la Clean Architecture / Hexagonal Architecture. Des mécanismes event-driven sont ajoutés pour le temps réel, les notifications, les exports et les traitements asynchrones.

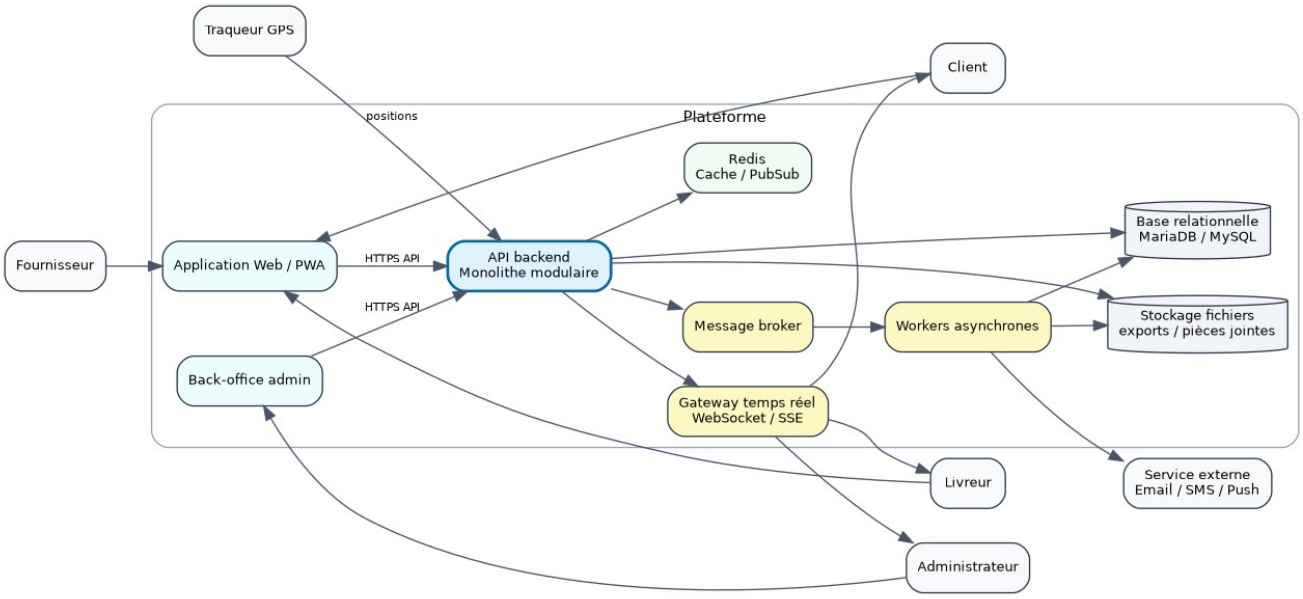
Ce choix est adapté au contexte : l'application existante est un MVP PHP sans framework, sans tests automatisés et difficile à maintenir. Une migration directe vers des microservices serait trop risquée. Le monolithe modulaire permet de restructurer d'abord le domaine métier, puis d'extraire plus tard certains modules si la charge l'exige.

Élément	Choix proposé
Backend	PHP 8.3+ avec Symfony ou Laravel, organisé en modules métier.
Frontend	Application Web / PWA pour clients, fournisseurs et livreurs ; back-office sécurisé pour administrateurs.
Base de données	MariaDB / MySQL conservée au départ, avec schéma nettoyé, migrations versionnées, index et contraintes.
Temps réel	WebSocket, Server-Sent Events ou Mercure pour les statuts et positions GPS.
Asynchrone	Message broker avec workers pour notifications, exports, statistiques et traitements longs.
Cache	Redis pour cache, sessions éventuelles, Pub/Sub et données temporaires.
Sécurité	RBAC, validation serveur, requêtes préparées ou ORM, protection CSRF/XSS, HTTPS, audit logs.
Tests	Tests unitaires, tests d'intégration, tests fonctionnels et pipeline CI/CD.

Diagramme de contexte

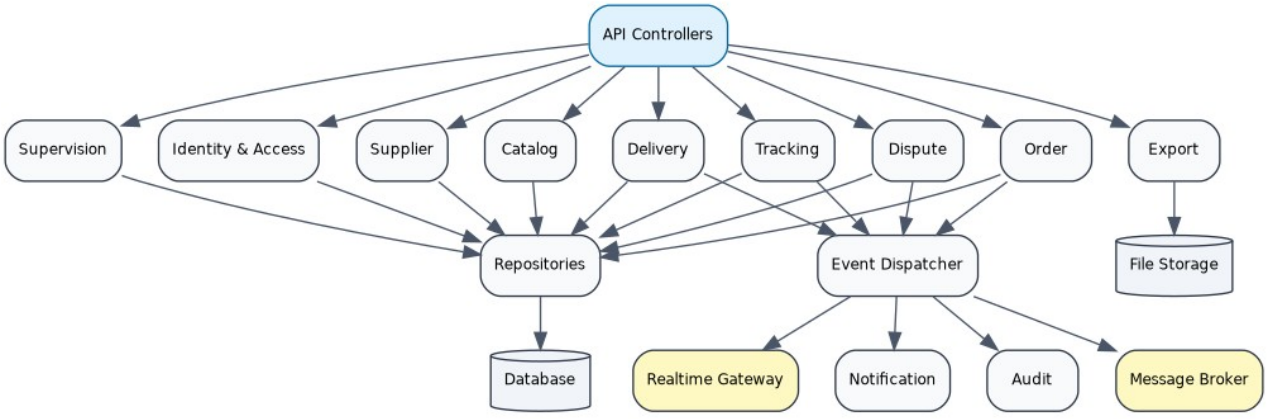


3. Diagramme de conteneurs



Conteneur	Responsabilité
Application Web / PWA	Interface client, fournisseur et livreur : catalogues, commandes, suivi, statuts.
Back-office administrateur	Supervision, litiges, exports, gestion sécurisée des données administratives.
API backend monolithe modulaire	Expose les API, applique les règles métier, valide les droits, orchestre les modules.
Gateway temps réel	Diffuse les positions GPS, changements de statuts et notifications en direct.
Message broker	Transporte les événements et tâches asynchrones.
Workers asynchrones	Traite notifications, exports, statistiques, emails et tâches longues.
Base relationnelle	Stocke utilisateurs, fournisseurs, catalogues, commandes, livraisons, positions, litiges, transactions.
Redis	Cache, Pub/Sub, sessions éventuelles et données temporaires.
Stockage fichiers	Stocke exports CSV/XLSX/PDF et pièces jointes de litiges.

4. Diagramme de composants



Module	Responsabilités principales
Identity & Access	Authentification, rôles, permissions, sessions ou tokens, contrôle d'accès par ressource.
Supplier	Gestion des fournisseurs et rattachement aux catalogues.
Catalog	Création des catalogues, articles, prix, disponibilités et désactivation d'articles.
Order	Création de commandes, validation des articles, calcul des montants, publication d'événements.
Delivery	Affectation aux livreurs, acceptation, refus, changement de statut, clôture.
Tracking	Réception des positions GPS, historisation, exposition de la dernière position connue.
Dispute	Création, suivi, assignation et résolution des litiges.
Supervision	Indicateurs : clients, livreurs, transactions, volume de fret, activité globale.
Export	Génération des exports CSV, XLSX et PDF.
Notification	Envoi de notifications aux clients, livreurs et administrateurs.
Audit	Journalisation des actions sensibles et accès administrateur.

5. Modèle de données cible

Table	Rôle
users	Comptes clients, fournisseurs, livreurs et administrateurs.
roles	Rôles et permissions applicatives.
suppliers	Informations propres aux fournisseurs.
couriers	Informations propres aux livreurs.
catalogs	Catalogues publiés par les fournisseurs.
catalog_items	Articles, prix et disponibilités.
orders	Commandes passées par les clients.
order_items	Lignes de commande.
deliveries	Livraisons liées aux commandes.
delivery_positions	Positions GPS historisées.
disputes	Litiges ouverts sur commandes ou livraisons.
dispute_messages	Messages et pièces liées aux litiges.
transactions	Volumes financiers et transactions.
audit_logs	Historique des actions sensibles.
exports	Fichiers exportés et traces de génération.
notifications	Notifications envoyées ou en attente.

6. Stratégie d'évolution et de migration des données

Étape	Actions
Audit de l'existant	Analyser les tables, flux métier, dépendances PHP, données critiques, failles visibles et cas d'utilisation.
Stabilisation	Sauvegarde complète, correction des failles critiques, logs, documentation des parcours, tests de non-régression minimum.
Schéma cible	Créer un modèle relationnel propre avec clés étrangères, index, contraintes, statuts contrôlés et migrations versionnées.
Migration à blanc	Exporter, nettoyer, mapper, importer et comparer les volumes de données en environnement de test.
Strangler Fig Pattern	Remplacer progressivement les anciens modules PHP par les nouveaux modules : Auth, Commandes,

	Livraisons, Litiges, Supervision.
Mise en production	Déploiement staging, sauvegarde avant migration, plan de rollback, monitoring renforcé et validation post-déploiement.

7. Évolutivité et mise à l'échelle

Horizon	Mesures
Court terme	Monolithe modulaire, cache Redis, workers asynchrones, indexation SQL, WebSocket/SSE.
Moyen terme	Scalabilité horizontale de l'API, load balancer, workers multiples, réplicas de lecture, stockage externe.
Long terme	Extraction possible de Tracking, Notification, Export ou Supervision en services indépendants si la charge le justifie.